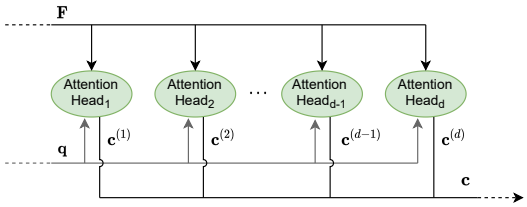

Fig. 9. An illustration of multi-head attention.

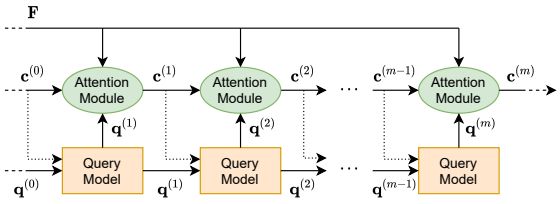

Fig. 10. An example illustration of multi-hop attention. Solid arrows represent the base multi-hop model structure, while dotted arrows represent optional connections.

learn to focus on different parts of the inputs, allowing the model to attend to more information. For example, when training a machine translation model, one attention head can learn to focus on which nouns (e.g., student, car, apple) do certain verbs (e.g., walking, driving, buying) refer to, while another attention head learns to focus on which nouns refer to certain pronouns (e.g., he, she, it) [13]. Each head will also create its own vector of attention scores $\boldsymbol{e}^{(j)} = [e_1^{(j)}, \ldots, e_{n_f}^{(j)}] \in \mathbb{R}^{n_f}$, and a corresponding vector of attention weights $\boldsymbol{a}^{(j)} = [a_1^{(j)}, \ldots, a_{n_f}^{(j)}] \in \mathbb{R}^{n_f}$. As can be expected, each attention model produces its own context vector $\boldsymbol{c}^{(j)} \in \mathbb{R}^{d_v}$, as follows:

$$\underset{d_v \times 1}{\boldsymbol{c}^{(j)}} = \sum_{l=1}^{n_f} \underset{1 \times 1}{a_l^{(j)}} \times \underset{d_v \times 1}{\boldsymbol{v}_l^{(j)}}. \tag{39}$$

The goal is still to create a single context vector as output of the attention model. As such, the context vectors produced by the individual attention heads are concatenated into a single vector. Afterwards, a linear transformation is applied using the weight matrix $\boldsymbol{W}_O \in \mathbb{R}^{d_c \times d_v d}$ to make sure the resulting context vector $\boldsymbol{c} \in \mathbb{R}^{d_c}$ has the desired dimension. This calculation is presented in (40). The dimension d_c can be pre-specified by, for example, setting it equal to d_v, so that the context vector dimension is unchanged.

$$\underset{d_c \times 1}{\boldsymbol{c}} = \underset{d_c \times d_v d}{\boldsymbol{W}_O} \times \text{concat}(\underset{d_v \times 1}{\boldsymbol{c}^{(1)}}, ..., \underset{d_v \times 1}{\boldsymbol{c}^{(d)}}). \tag{40}$$

Multi-head attention processes multiple attention modules in parallel, but attention modules can also be implemented sequentially to iteratively adjust the context vectors. Each of these attention modules are referred to as "repetitions" or "rounds" of attention. Such attention architectures are referred to as **multi-hop attention models**, also known as **multi-step attention models**. An important note to consider is the fact that multi-hop attention is a mechanism that has been proposed in various forms throughout various works. While the mechanism always involves multiple rounds of attention, the multi-hop implementation proposed in [84] differs from the mechanism proposed in [85] or [86]. Another interesting example is [87], where a "multi-hop" attention model is proposed that would actually be considered alternating co-attention in this survey, as explained in Subsection 3.1.1.

We present a general form of multi-hop attention that is largely a generalization of the techniques introduced in [85] and [88]. Fig. 10 provides an example implementation of a multi-hop attention mechanism. The general idea is to iteratively transform the query, and use the query to transform the context vector, such that the model can extract different information in each step. Remember that a query is similar to a literal question. As such, one can interpret the transformed queries as asking the same question in a different manner or from a different perspective, similarly to the queries in multi-head attention. The query that was previously denoted by $\boldsymbol{q}$ is now referred to as the initial query, and is denoted by $\boldsymbol{q}^{(0)}$. At hop s, the current query $\boldsymbol{q}^{(s)}$ is transformed into a new query representation $\boldsymbol{q}^{(s+1)}$, possibly using the current context vector $\boldsymbol{c}^{(s)}$ as another input, and some transformation function transform():

$$\underset{d_q \times 1}{\boldsymbol{q}^{(s+1)}} = \text{transform}(\underset{d_q \times 1}{\boldsymbol{q}^{(s)}}, \underset{d_v \times 1}{\boldsymbol{c}^{(s)}}). \tag{41}$$

For the specific form of the transformation function transform(), [85] proposes to use a mechanism similar to self-attention. Essentially, the queries used by the question answer matching model proposed in [85] were originally based on a set of feature vectors extracted from a question. [85] also defines the original query $\boldsymbol{q}^{(0)}$ as the unweighted average of these feature vectors. At each hop s, attention can be calculated on these feature vectors using the previous query $\boldsymbol{q}^{(s)}$ as the query in this process. The resulting context vector of this calculation is the next query vector. Using the context vector $\boldsymbol{c}^{(s)}$ instead of $\boldsymbol{q}^{(s)}$ as the query for this process is also a possibility, which is similar to the *LCR-Rot-hop* model proposed in [43] and the multi-step model proposed in [88]. Such a connection is represented by the dotted arrows in Fig. 10. The transformation mechanism uses either the $\boldsymbol{q}^{(s)}$ or the context vector $\boldsymbol{c}^{(s)}$ as query, but a combination via concatenation is also possible.

Each query representation is used as input for the attention module to compute attention on the columns of the feature matrix $\boldsymbol{F}$, as seen previously. One main difference, however, is that the context vector $\boldsymbol{c}^{(s)}$ is also used as input, so that the actual query input for the attention model is the concatenation of $\boldsymbol{c}^{(s)}$ and $\boldsymbol{q}^{(s+1)}$. The adjusted attention score function is presented in (42). Note that the initial context vector $\boldsymbol{c}^{(0)}$ is predefined. One way of doing this is by setting it equal to the unweighted average of the value vectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{n_f} \in \mathbb{R}^{d_v}$ extracted from $\boldsymbol{F}$.

$$\underset{1 \times 1}{e_l^{(s)}} = \text{score}(\text{concat}(\underset{d_q \times 1}{\boldsymbol{q}^{(s+1)}}, \underset{d_v \times 1}{\boldsymbol{c}^{(s)}}), \underset{d_k \times 1}{\boldsymbol{k}_l}). \tag{42}$$

An alignment function and the value vectors are then used to produce the next context vector $\boldsymbol{c}^{(s+1)}$. One must note that in [85], the weights used in each iteration are the same weights, meaning that the number of parameters do not scale with the number of repetitions. Yet, using multiple hops with different weight matrices can also be viable, as shown by the Transformer model [13] and in [88]. It may be

difficult to grasp why $\boldsymbol{c}^{(s)}$ is part of the query input for the attention model. Essentially, this technique is closely related to self-attention in the sense that, in each iteration, a new context representation is created from the feature vectors and the context vector. The essence of this mechanism is that one wants to iteratively alter the query and the context vector, while attending to the feature vectors. In the process, the new representations of the context vector absorb more different kinds of information. This is also the main difference between this type of attention and multi-head attention. Multi-head attention creates multiple context vectors from multiple queries and combines them to create a final context vector as output. Multi-hop attention iteratively refines the context vector by incorporating information from the different queries. This does have the disadvantage of having to calculate attention sequentially.

Interestingly, due to the variations in which multi-hop attention has been proposed, some consider the Transformer model's encoder and decoder to consist of several single-hop attention mechanisms [84] instead of being a multi-hop model. However, in the context of this survey, we consider the Transformer model to be an alternative form of the multi-hop mechanism, as the features matrix $\boldsymbol{F}$ is not directly reused in each step. Instead, $\boldsymbol{F}$ is only used as an input for the first hop, and is transformed via self-attention into a new representation. The self-attention mechanism uses each feature vector in $\boldsymbol{F}$ as a query, resulting in a matrix of context vectors as output of each attention hop. The intermediate context vectors are turned into matrices and represent iterative transformations of the matrix $\boldsymbol{F}$, which are used in the consecutive steps. Thus, the Transformer model iteratively refines the features matrix $\boldsymbol{F}$ by extracting and incorporating new information.

When dealing with a classification task, another idea is to use a different query for each class. This is the basic principle behind **capsule-based attention** [89], as inspired by the capsule networks [90]. Suppose we have the feature vectors $\boldsymbol{f}_1, \ldots, \boldsymbol{f}_{n_f} \in \mathbb{R}^{d_f}$, and suppose there are are d_y classes that the model can predict. Then, a capsule-based attention model defines a capsule for each of the d_y classes that each take as input the feature vectors. Each capsule consists of, in order, an attention module, a probability module, and a reconstruction module, which are depicted in Fig. 11. The attention modules all use self-attentive queries, so each module learns its own query: "Which feature vectors are important to identify this class?". In [89], a self-attentive multiplicative score function is used for this purpose:

$$\underset{1\times 1}{e_{c,l}} = \underset{1\times d_k}{\boldsymbol{q}_c^T} \times \underset{d_k\times 1}{\boldsymbol{k}_l} \ , \tag{43}$$

where $e_{c,l} \in \mathbb{R}^1$ is the attention score for vector l in capsule c, and $\boldsymbol{q}_c \in \mathbb{R}^{d_k}$ is a trainable query for capsule c, for $c = 1, \ldots, d_y$. Each attention module then uses an alignment function, and uses the produced attention weights to determine a context vector $\boldsymbol{c}_c \in \mathbb{R}^{d_v}$. Next, the context vector $\boldsymbol{c}_c$ is fed through a probability layer consisting of a linear transformation with a sigmoid activation function:

$$\underset{1\times 1}{p_c} = \text{sigmoid}(\underset{1\times d_v}{\boldsymbol{w}_c^T} \times \underset{d_v\times 1}{\boldsymbol{c}_c} + \underset{1\times 1}{b_c}), \tag{44}$$

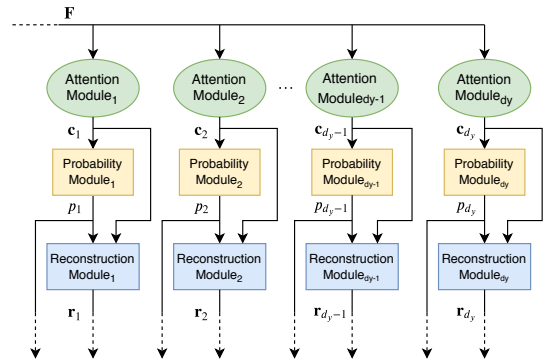

Fig. 11. An illustration of capsule-based attention.

where $\boldsymbol{w}_c \in \mathbb{R}^{d_v}$ and $b_c \in \mathbb{R}^1$ are trainable capsule-specific weights parameters, and $p_c \in \mathbb{R}^1$ is the predicted probability that the correct class is class c. The final layer is the reconstruction module that creates a class vector representation. This representation $\boldsymbol{r}_c \in \mathbb{R}^{d_v}$ is determined by simply multiplying the context vector $\boldsymbol{c}_c$ by the probability p_c:

$$\underset{d_v\times 1}{\boldsymbol{r}_c} = \underset{1\times 1}{p_c} \times \underset{d_v\times 1}{\boldsymbol{c}_c} \ . \tag{45}$$

The capsule representation is used when training the model. First of all, the model is trained to predict the probabilities $p_1, \ldots, p_{d_y}$ as accurately as possible compared to the true values. Secondly, via a joint loss function, the model is also trained to accurately construct the capsule representations $\boldsymbol{r}_1, \ldots, \boldsymbol{r}_{d_y}$. A features representation $\boldsymbol{f} \in \mathbb{R}^{d_f}$ is defined which is simply the unweighted average of the original feature vectors. The idea is to train the model such that vector representations from capsules that are not the correct class differ significantly from $\boldsymbol{f}$ while the representation from the correct capsule is very similar to $\boldsymbol{f}$. A dot-product between the capsule representations and the features representation is used in [89] as a measure of the distance between the vectors. Note that d_v must equal d_f in this case, otherwise the vectors would have incompatible dimensions. Interestingly, since attention is calculated for each class individually, one can track which specific feature vectors are important for which specific class. In [89], this idea is used to discover which words correspond to which sentiment class.

The number of tasks that can make use of multiple queries is substantial, due to how general the mechanisms are. As such, the techniques described in this section have been extensively explored in various domains. For example, multi-head attention has been used for speaker recognition based on audio spectrograms [91]. In [92], multi-head attention is used for recommendation of news articles. Additionally, multi-head attention can be beneficial for graph attention models as well [83]. As for multi-hop attention, quite a few papers have been mentioned before, but there are still many other interesting examples. For example, in [93], a multi-hop attention model is proposed for medication recommendation. Furthermore, practically every Transformer model makes use of both multi-head and multi-hop attention. The Transformer model has been extensively explored in various domains. For example, in [94], a Transformer model is implemented for image cap-

tioning. In [95], Transformers are explored for medical image segmentation. In [96], a Transformer model is used for emotion recognition in text messages. A last example of an application of Transformers is [17], which proposes a Transformer model for recommender systems. In comparison with multi-head and multi-hop attention, capsule-based attention is arguably the least popular of the mechanisms discussed for the multiplicity of queries. One example is [97], where an attention-based capsule network is proposed that also includes a multi-hop attention mechanism for the purpose of visual question answering. Another example is [98], where capsule-based attention is used for aspect-level sentiment analysis of restaurant reviews.

The multiplicity of queries is a particularly interesting category due to the Transformer model [13], which combines a form of multi-hop and multi-head attention. Due to the initial success of the Transformer model, many improvements and iterations of the model have been produced that typically aim to improve the predictive performance, the computational efficiency, or both. For example, the Transformer-XL [99] is an extension of the original Transformer that uses a recurrence mechanism to not be limited by a context window when processing the outputs. This allows the model to learn significantly longer dependencies while also being computationally more efficient during the evaluation phase. Another extension of the Transformer is known as the Reformer model [100]. This model is significantly more efficient computationally, by means of locality-sensitive hashing, and memory-wise, by means of reversible residual layers. Such computational improvements are vital, since one of the main disadvantages of the Transformer model is the sheer computational cost due to the complexity of the model scaling quadratically with the amount of input feature vectors. The Linformer model [101] manages to reduce the complexity of the model to scale linearly, while achieving similar performance as the Transformer model. This is achieved by approximating the attention weights using a low-rank matrix. The Lite-Transformer model proposed in [102] achieves similar results by implementing two branches within the Transformer block that specialize in capturing global and local information. Another interesting Transformer architecture is the Synthesizer [103]. This model replaces the pairwise self-attention mechanism with "synthetic" attention weights. Interestingly, the performance of this model is relatively close to the original Transformer, meaning that the necessity of the pairwise self-attention mechanism of the Transformer model may be questionable. For a more comprehensive overview of Transformer architectures, we refer to [104].

4 Evaluation of Attention Models

In this section, we present various types of evaluation for attention models. Firstly, one can evaluate the structure of attention models using the taxonomy presented in Section 3. For such an analysis, we consider the attention mechanism categories (see Fig. 3) as orthogonal dimensions of a model. The structure of a model can be analyzed by determining which mechanism a model uses for each category. Table 3 provides an overview of attention models found in the literature with a corresponding analysis based on the attention mechanisms the models implement.

Secondly, we discuss various techniques for evaluating the performance of attention models. The performance of attention models can be evaluated using **extrinsic** or **intrinsic** performance measures, which are discussed in Subsections 4.1 and 4.2, respectively.

4.1 Extrinsic Evaluation

In general, the performance of an attention model is measured using **extrinsic performance measures**. For example, performance measures typically used in the field of natural language processing are the BLEU [107], METEOR [108], and Perplexity [109] metrics. In the field of audio processing, the Word Error Rate [110] and Phoneme Error Rate [111] are generally employed. For general classification tasks, error rates, precision, and recall are generally used. For computer vision tasks, the PSNR [112], SSIM [113], or IoU [114] metrics are used. Using these performance measures, an attention model can either be compared to other state-of-the-art models, or an ablation study can be performed. If possible, the importance of the attention mechanism can be tested by replacing it with another mechanism and observing whether the overall performance of the model decreases [105], [115]. An example of this is replacing the weighted average used to produce the context vector with a simple unweighted average and observing whether there is a decrease in overall model performance [35]. This ablation method can be used to evaluate whether the attention weights can actually distinguish important from irrelevant information.

4.2 Intrinsic Evaluation

Attention models can also be evaluated using attention-specific **intrinsic performance measures**. In [4], the attention weights are formally evaluated via the Alignment Error Rate (AER) to measure the accuracy of the attention weights with respect to annotated attention vectors. [116] incorporates this idea into an attention model by supervising the attention mechanism using gold attention vectors. A joint loss function consisting of the regular task-specific loss and the attention weights loss function is constructed for this purpose. The gold attention vectors are based on annotated text data sets where keywords are hand-labelled. However, since attention is inspired by human attention, one could evaluate attention models by comparing them to the attention behaviour of humans.

4.2.1 Evaluation via Human Attention

In [117], the concept of **attention correctness** is proposed, which is a quantitative intrinsic performance metric that evaluates the quality of the attention mechanism based on actual human attention behaviour. Firstly, the calculation of this metric requires data that includes the attention behaviour of a human. For example, a data set containing images with the corresponding regions that a human focuses on when performing a certain task, such as image captioning. The collection of regions focused on by the human is referred to as the ground truth region. Suppose an attention model attends to the n_f feature vectors $\boldsymbol{f}_1, \ldots, \boldsymbol{f}_{n_f} \in \mathbb{R}^{d_f}$. Feature vector $\boldsymbol{f}_i$ corresponds to region R_i of the given